

请通过以下程序理解变量和信号的区别

```
library IEEE;
use IEEE.std_logic_1164.all;

entity sig_var is
port (x, y, z : in std_logic;
      res1, res2 : out std_logic);
end entity sig_var;
```

```
architecture dataflow of sig_var is
signal sig_s1, sig_s2 : std_logic;
begin
  proc1: process (x, y, z) is
  variable var_s1, var_s2 : std_logic;
  begin
    L1: var_s1 := x and y;
    L2: var_s2 := var_s1 xor z;
    L3: res1 <= var_s1 nand var_s2;
  end process proc1;

  proc2: process (x, y, z) is
  begin
    L1: sig_s1 <= x and y;
    L2: sig_s2 <= sig_s1 xor z;
    L3: res2 <= sig_s1 nand sig_s2;
  end process proc2;
end architecture dataflow;
```

1. 假设 x、y、z 信号为初始值为 0、占空比为 50% 的周期方波，其中 x 信号的周期为 80 ns，y 信号的周期为 40 ns，z 信号的周期为 20 ns。
2. 请自行推导 res1 和 res2 的输出波形。
3. 请设计 testbench 文件进行仿真以验证你的结果。

推导 res1 和 res2 的输出波形：

信号

0 ~ 10 ns x=0 y=0 z=0
~~sig-s1~~ sig-s1=0 sig-s2=uninitialized res=uninitialized

10 ~ 20 ns x=0 y=0 z=1
 sig-s1=0 sig-s2=1 res2=1

20 ~ 30 ns x=0 y=1 z=0
 sig-s1=0 sig-s2=0 res2=1

30 ~ 40 ns x=0 y=1 z=1
 sig-s1=0 sig-s2=1 res2=1

40 ~ 50 ns x=1 y=0 z=0
 sig-s1=0 sig-s2=0 res2=1

50 ~ 60 ns x=1 y=0 z=1
 sig-s1=0 sig-s2=1 res2=1

60 ~ 70 ns x=1 y=1 z=0
 sig-s1=1 sig-s2=0 res2=1

70 ~ 80 ns x=1 y=1 z=1
 sig-s1=1 sig-s2=0 res2=1

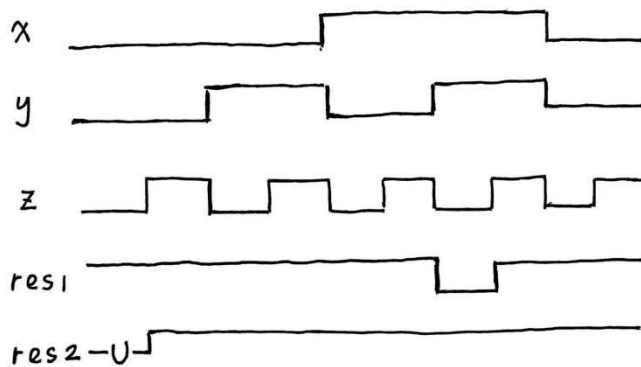
80 ~ 90 ns x=0 y=0 z=0
 sig-s1=0 sig-s2=1 res2=1

90 ~ 100 ns x=0 y=0 z=1
 sig-s1=0 sig-s2=1 res2=1

:

变量

x	y	z	var-s1	var-s2	res1
0	0	0	0	0	1
0	0	1	0	1	1
0	1	0	0	0	1
0	1	1	0	1	1
1	0	0	0	0	1
1	0	1	0	1	1
1	1	0	1	1	0
1	1	1	1	0	1



testbench 代码:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity sig_var_tb is
-- Port ( );
end sig_var_tb;

-- 结构体
architecture tb of sig_var_tb is
-- 元件声明
component sig_var is
    Port ( x : in STD_LOGIC;
           y : in STD_LOGIC;
           z : in STD_LOGIC;
           res1 : out STD_LOGIC;
           res2 : out STD_LOGIC);
end component;

-- 信号声明
signal x_i, y_i, z_i, r1_o, r2_o : std_logic := '0';
-- 常数声明
constant period : time := 10ns;

begin

    uut: sig_var port
        map ( x => x_i,
              y => y_i,
              z => z_i,
              res1 => r1_o,
```

```

        res2 => r2_o);

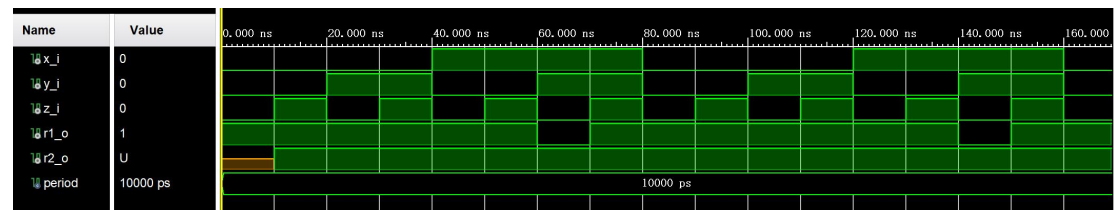
    x_i <= not x_i after period*8;
    y_i <= not y_i after period*4;
    z_i <= not z_i after period*2;

end tb;

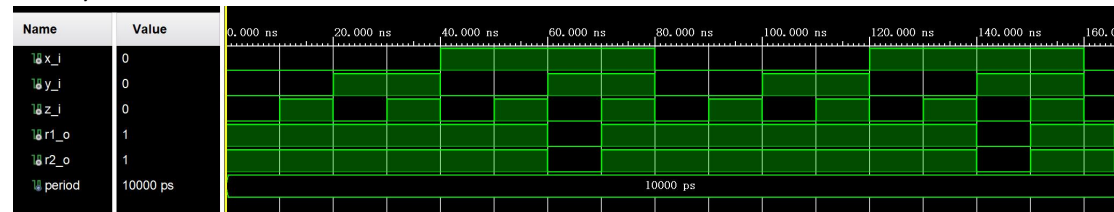
```

仿真结果：

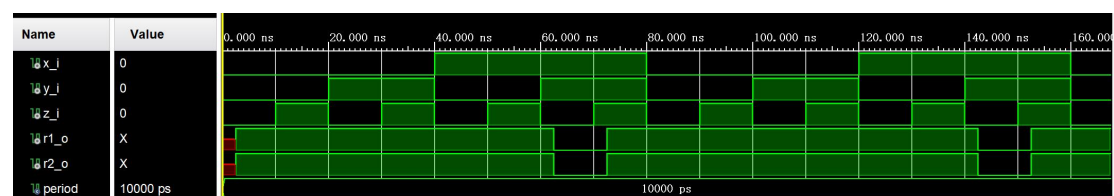
Behavioral simulation



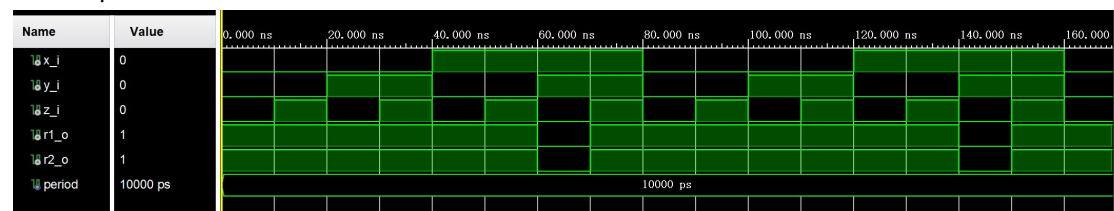
Post-synthesis simulation - functional



Post-synthesis simulation - timing



Post-implementation simulation - functional



Post-implementation simulation - timing

